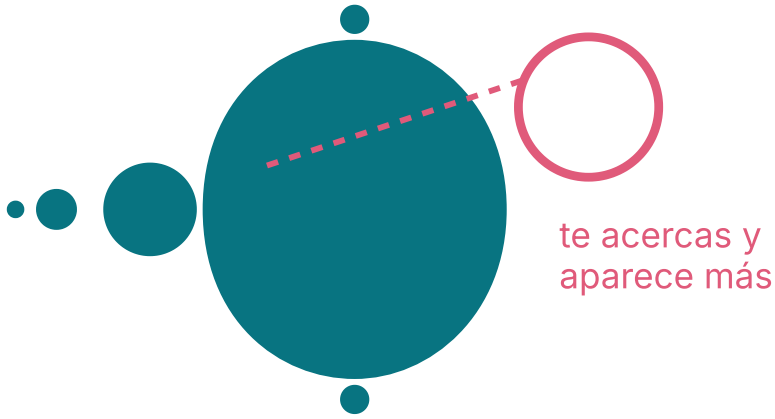
SYMULA · MATEMÁTICA

¿Un dibujo que nunca se termina?



Cada punto de la pantalla juega un juego: toma un número z que empieza en cero, lo **multiplica por sí mismo** y le suma el número c del punto. Luego lo repite, una y otra vez.

Si z se queda chiquito para siempre, el punto se pinta negro: es parte del conjunto de Mandelbrot. Si crece y se escapa, se pinta de un color que dice cuántas vueltas tardó en escaparse.

Un poco de matemática. Con $c = 1$: $0 \to 0 \cdot 0 + 1 = 1 \to 1 \cdot 1 + 1 = 2 \to 2 \cdot 2 + 1 = 5 \to 26 \to 677 \ldots$ ¡se escapa!
Con $c = -1$: $0 \to -1 \to 0 \to -1 \to 0 \ldots$ se queda dando vueltas para siempre: ese punto es negro.

Desafío. Con $c = -2$, ¿el número se escapa o se queda?

Respuesta: $0 \to -2 \to 2 \to 2 \to 2 \ldots$ se queda en 2 para siempre: el punto −2 es negro (es la punta de la figura).

Pruébalo: acércate al borde de la figura negra. ¿Encuentras una figura pequeñita igual a la grande?

Fractales

SyMula · Fórmulas con las que se calcula el simulador (fractales.html)

La regla

Para cada número complejo c del plano se itera

$$z_{n+1} = z_n^{\,d} + c, \qquad d \in \{2, 3, 4\}.$$

Mandelbrot: $z_0 = 0$ y c es el punto de la pantalla. **Julia:** c es fijo (se elige con los controles o es el centro que se estaba mirando) y z_0 es el punto de la pantalla. Un punto escapa cuando $|z_n|^2 > 2^{16}$; si en N iteraciones no escapa, se pinta negro.

Color suave

Para que no se vean escalones se usa el conteo fraccionario

$$\nu = n + 1 - \frac{\ln(\ln|z_n|)}{\ln d},$$

y el color sale de una paleta cíclica en la posición $t = \rho\,0{,}32\,\ln(1 + \nu - \nu_{\text{mín}}) + \text{giro}$, donde ρ es el "ritmo de color" y $\nu_{\text{mín}}$ el menor valor de la imagen (así hay contraste a cualquier zoom).

Zoom sin límite práctico: perturbación

Un número normal tiene unas 16 cifras, así que pasado un zoom de 10^{14} todos los píxeles serían el mismo número. Por eso solo el **centro** se calcula con enteros grandes (punto fijo con P bits, $P \approx -\log_2(\text{tamaño del píxel}) + 48$): es la órbita de referencia Z_n. Cada píxel guarda solo su diferencia $\delta_n = z_n - Z_n$, que sí cabe en un número normal:

$$\delta_{n+1} = (Z_n + \delta_n)^d - Z_n^{\,d} + \delta c = \delta_n \sum_{k=1}^{d} \binom{d}{k} Z_n^{\,d-k}\,\delta_n^{\,k-1} + \delta c.$$

Para $d = 2$: $\delta_{n+1} = 2Z_n\delta_n + \delta_n^2 + \delta c$. En Julia $\delta c = 0$ y δ_0 es la distancia al centro.

Cambio de referencia. Si $|Z_n + \delta_n| < |\delta_n|$, o si se acaba la referencia, se continúa con la órbita que parte de 0 ($W_0 = 0$, $W_{m+1} = W_m^{\,d} + c$) tomando $\delta = z$ y $m = 0$. Así se evitan los errores ("glitches") del método. El tope es un zoom de unas 10^{280} veces, cerca del número más chico que se puede guardar.

Las iteraciones crecen con el zoom: $N = 2^k\,(100 + 50\log_2 \text{zoom})$, con k el control "Iteraciones" (hasta 50 000).

Cómo está hecho el archivo HTML

- Es **un solo archivo** con HTML, CSS y JavaScript. El navegador lo ejecuta tal cual.
- **La vista** guarda el centro como dos enteros grandes (`BigInt`) y el tamaño del píxel. `pan()` y `zoomAt()` los cambian al arrastrar, pellizcar o usar la rueda.
- **La referencia:** `orbit()` itera el centro con enteros grandes; `refs()` la reutiliza si nada cambió.
- **El cálculo:** `kernel()` recorre filas de píxeles con la fórmula de perturbación. Se reparte entre varios núcleos del equipo con *Web Workers*, de 8 filas a la vez.
- **Primero rápido, después nítido:** mientras te mueves se calcula una imagen chica (su tamaño se ajusta a lo que alcanza el equipo) y la última imagen se estira; al soltar se calcula la imagen a resolución completa.
- **El color:** `paint()` convierte cada conteo en color con la paleta, sin recalcular el fractal.

```
if (d === 2){                              // δ' = 2Zδ + δ² + δc
  nr = 2*(Zr*dr - Zi*di) + dr*dr - di*di + cr;
  ni = 2*(Zr*di + Zi*dr) + 2*dr*di + ci;
}
if (r2 < dr*dr + di*di || m >= len - 1){   // cambio de referencia
  dr = zr; di = zi; ref = B; len = lenB; m = 0;
}
```

> Comprobado en `pruebas/fractales.js`: los píxeles coinciden con la iteración directa a zoom 1, 10^{7}, 10^{9} y 10^{31} (esta última contra enteros grandes, punto por punto).